

2.6 - ArrayLists

The designers of Java had many choices to make when they implemented the ArrayList data structure. Each of these choices could have an impact on how fast list operations perform. To help them make the right choices they looked at the ways that people would most commonly use the ArrayList (which we shall call “list” for short) data structure, and they optimized their implementation of a list so that the most common operations were very fast. Of course they also tried to make the less common operations fast, but when a trade-off had to be made the performance of a less common operation was often sacrificed in favor of the more common operation.

Two common operations are indexing and assigning to an index position. Both of these operations take the same amount of time no matter how large the list becomes. When an operation like this is independent of the size of the list, it is $O(1)$.

Another very common programming task is to grow an ArrayList by using the `add` method, which is $O(n)$, but in a very special way that we will examine in **Section 8.2**.

Here is a table of common ArrayList operations and their Big O efficiency:

Table 2.6.1.

Operation	Big O Efficiency
<code>get(index)</code>	$O(1)$
<code>set(index)</code>	$O(1)$
<code>add n items</code>	$O(n)$
<code>remove(index)</code>	$O(n)$
<code>indexOf()</code>	$O(n)$

Let’s examine the `remove` method a bit more closely. The $O(n)$ is a *worst-case* number. When we remove the first number in an ArrayList, all the remaining numbers have to be moved left one position. If we remove the last number in an ArrayList, that is a *best-case* situation, and is $O(1)$, as no elements need to be moved.

Here's the results of a program that removes the first item from an ArrayList 1000 times and compares that time to the time required to remove the last item 1000 times.

```
2,000,000 items
Remove first time: 0.3152410 sec
Remove last time: 0.0000029 sec

1,000,000 items
Remove first time: 0.0858035 sec
Remove last time: 0.0000022 sec

100,000 items
Remove first time: 0.0055089 sec
Remove last time: 0.0000017 sec
```

As you see, the worst case time is proportional to the size of the list; the best case time is relatively unchanged.

[Listing 2.6.2](#) is the program that we used, and there are some important things to note:

Listing 2.6.2.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 import java.util.Scanner;
2 import java.util.ArrayList;
3
4 public class RemoveItem {
5
6
7     public static double timeRemove(int len, boolean removeF:
8         final int N_TRIALS = 25;
9         final int N_IGNORE = 20; // let system stabilize
10        long totalTime = 0;
11
12        ArrayList<Integer> list = new ArrayList<>(len);
13
14        for (int trial = 0; trial < N_TRIALS; trial++) {
15            int start = list.size();
16            for (int i = start; i < len; i++) {
17                list.add(i);
18            }
19            System.gc();
20
21            long startTime = System.nanoTime();
22            for (int count = 0; count < 1000; count++) {
23                if (removeFirst) {
24                    list.remove(0);
25                } else {
26                    list.remove(list.size() - 1);
27                }
28            }
29
30            if (trial >= N_IGNORE) {
```

In lines 8 and 9, we will run each test 25 times and average only the last 5 (in line 35), giving the cache time to stabilize as we saw in [Listing 2.2.3](#).

We allocate the ArrayList once before the trials, in line 12. This will be an empty list, so we have to fill it in lines 15-18. This has to be done at every trial, because each trial removes 1000 items.

Line 19 introduces a very important concept. When Java allocates and re-allocates memory, it is possible to have an area of memory that is no longer referred to. The Java Virtual Machine periodically goes through memory and collects these unused areas so they can be re-used. This process is called **garbage collection**, and it can take a fair amount of time. We don't want this to happen while we are in our timing loop, so we call `System.gc()` to explicitly invoke garbage collection. This is not a sure-fire cure; the documentation for `gc` says: "There is no guarantee that this effort will recycle any particular number of unused objects, reclaim any particular amount of space, or complete at any particular time, if at all, before the method returns or ever." Nonetheless, calling `System.gc()` is our best effort to avoid garbage collection at inopportune times.

Finally, because the inner loop repeats a thousand times, it is also important to point out that the list is decreasing in size by one each time through the loop. But even in the shortest list of 100,000 elements, this only reduces the overall size by 1% (and only 0.05% in the case of the 2,000,000 element list).

You have attempted 1 of 2 activities on this page.



New: MCP for Claude & Cursor. Your AI gets the error, trace, deploy

and code. No vibe-
debugging.

www.appsignal.com

Ads by EthicalAds